

SchwarMAX — Agent's Guide

A hand-off document for an LLM helping a new user get started with the `schwarmax_pkg` codebase. Read this in full before your first task.

1. What SchwarMAX is

JAX-based Schwarzschild dynamical modelling framework for barred galaxies. Given IFU-style stellar kinematics (Voronoi-binned surface density Σ , mean line-of-sight velocity V , dispersion σ , and Gauss-Hermite moments h_1 – h_4) it reconstructs the joint constraint on:

- an NFW dark-matter halo,
- a Miyamoto-Nagai stellar disc,
- a T3 ellipsoidal bar + V4 ovoid bulge (Dehnen 2023),
- three viewing angles (α, β, γ), a mass-to-light ratio, and a bar pattern speed Ω .

The forward model integrates orbits in the bar's rotating frame, projects each orbit into IFU bins and Gauss-Hermite moments, then solves a non-negative-least-squares (NNLS) problem for the orbital weights that best reproduce the data. That inner NNLS + orbit-integration block runs **for every parameter proposal in an MCMC** (typically BlackJAX adaptive random-walk Metropolis on GPU).

The public interface lives in `schwarmax_pkg/` at the root of this repo. Everything is pure functions of `(theta, dict_data)` — no mutable globals, no runtime side effects.

2. How to use this document

If you are an LLM agent working with a user who wants to *use* SchwarMAX:

1. Read Sections 3–5 to internalise the package layout and parameter conventions.
2. Skim Sections 6–7 (entry points + workflows) — refer back when the user asks how to do something.
3. **Read Section 8 (pitfalls) carefully.** Every item is a real trap that has caused real bugs. Do not skip.
4. When you need to write code, prefer the pattern from Section 7 over inventing your own. If you have to invent, verify signatures with `Read` on the referenced file:line — do not guess.
5. The `example/` folder is the canonical end-to-end demonstration (Sections 9–10). Point the user there.

If the user asks something you can't answer from this document, read the relevant source file directly. Every function is short (< 200 lines) and well-named.

3. Directory layout

```
schwarmax_pkg/
├── constants.py
├── utils.py
├── potentials.py
├── dehnen_bar.py
├── jeans.py
├── ghmoments.py
├── integrates.py
├── nnls.py
├── model.py
└──
# <-- put on sys.path for `from utils import ...`
# G, KPCGYR_TO_KMS, EPSILON, HYPERGEOM tables
# data loader, param converters, rotation curves
# NFW, MN potential + density
# T3, T4, V4 potentials + densities
# Jeans-moment initial-condition sampler
# Gauss-Hermite basis + h_to_V_sigma
# adaptive leapfrog integrator (vmap+chunked)
# ADMM NNLS solver, single + bootstrap
# build_model, get_model_with_orbit,
# model_deltaChi2_jackknife ← main entry
```

```

└─ likelihood.py          # model_likelihood, calculate_delta_chi2
└─ CylindricalSpline.py  # cyl-spline potential expansion (bar branch)
└─ CubicSpline.py        # helper for CylindricalSpline
└─ example/
  └─ example_schwarmax.ipynb  # run an MCMC
  └─ example_analysis.ipynb   # analyse a finished MCMC
  └─ mcmc_checkpoint.pkl      # 12-D BlackJAX chain (1000×32×12)
  └─ mock_Nbody_galaxy_beta65_gamma45_D50.pkl  # Voronoi kinematic data
  └─ Bar_model_TG21/model/    # N-body snapshot + AGAMA .ini
└─ docs/
  └─ LLM_GUIDE.md            # this file

```

Runtime dependencies: jax, jaxopt, numpy, scipy, matplotlib, corner, agama, astropy, blackjax (only if training an MCMC).

4. Parameter convention

The public parameter vector is **12-D**. Enforce this order everywhere:

slot	name	unit	notes
0	logM_10kpc	$\log_{10}(M_{\odot})$	NFW mass inside 10 kpc
1	logM_disc	$\log_{10}(M_{\odot})$	MN disc mass
2	logM_bar	$\log_{10}(M_{\odot})$	T3 bar mass = V4 bulge mass
3	logc_halo	$\log_{10}$	NFW concentration ($\Delta=200$, $\rho_c=277.54$)
4	logRs_disc	$\log_{10}(\text{kpc})$	MN scale length
5	logHs_disc	$\log_{10}(\text{kpc})$	MN scale height (= T3 b_bar)
6	logL_bar	$\log_{10}(\text{kpc})$	T3 half-length; $a_{\text{bar}} = L_{\text{bar}}/5$
7	alpha	radians	viewing angle α
8	beta	radians	viewing angle β
9	gamma	radians	viewing angle γ
10	log_mass_to_light	$\log_{10}(M/L)$	see pitfall §8.1
11	log_Omega_bar	$\log_{10}(\text{kpc/Gyr})$	multiply by KPCGYR_TO_KMS for km/s/kpc

Angles inside the parameter vector are **radians**. `build_model` receives the baryon dict with angles converted to **degrees**. The `unpack_params` helper in `example/example_analysis.ipynb` does this conversion for you (Section 7.3 below).

An **older 13-D convention** exists in the sibling `SchwarMAX/` codebase. `example_analysis.ipynb` handles both — see pitfall §8.1.

5. dict_data schema

`utils.load_data_bootstrap(path, filename, N_BOOTSTRAP=100, n_samples=5000)` loads a mock Voronoi-binned pickle and returns a dict with the keys below. **Do not hand-craft dict_data — always use this loader.**

Kinematic data + errors:

key	shape	meaning
-----	-------	---------

XY_density_data	(n_bins,)	surface density Σ (in luminosity units)
XY_density_data_err	(n_bins,)	1% relative error
V_data, V_data_err	(n_bins,)	V_los and its error
sigma_data, sigma_data_err	(n_bins,)	σ and its error
h1_data .. h4_data	(n_bins,)	Gauss-Hermite moments
h1_data_err .. h4_data_err	(n_bins,)	GH moment errors
v0, s	(n_bins,)	per-bin base-Gaussian centre and width
num_per_bin	(n_bins,)	pixel count per Voronoi bin
bin_mapping	(n_pixels,)	pixel-index $\rightarrow$ bin-index (+1 sentinel at end)
total_bins	scalar (int)	n_bins

Voronoi grid geometry:

key	shape	meaning
X_regular_grid, Y_regular_grid	(n_pixels,)	pixel centres (kpc)
X_minmax, Y_minmax	(2,)	grid extent
nX_nY	(2,)	pixel count per axis
dX, dY	scalar	pixel size

Rzphi integration grid (deterministic, built inside the loader):

key	shape	meaning
R_grid, z_grid, phi_grid	(Rzphi_n_tot,)	(R, z, ϕ) cell centres
R_minmax, z_minmax, phi_minmax	(2,)	ranges
Rzphi_n_tot	scalar	10·6·10 = 600 by default
Rzphi_n_grid	(3,)	(n_R, n_z, n_ ϕ) = (10, 6, 10)
dR, dz, dphi	scalars	cell widths
sample_for_integration	(1024, 3)	Sobol samples for sub-cell integration
sample_for_integration_XY	(1024, 3)	same for the XY grid

Bootstrap-perturbation normals (each has row 0 zeroed = unperturbed):

key	shape
XY_standard_normal	(N_BOOTSTRAP, n_bins)
h1_standard_normal .. h4_standard_normal	(N_BOOTSTRAP, n_bins)
V_standard_normal, sigma_standard_normal	(N_BOOTSTRAP, n_bins)

Initial-condition particles:

key	shape	meaning
-----	-------	---------

w0	(n_samples, 3)	(x, y, z) draws from the analytic disc
----	-------------------	---

n_samples defaults to 5,000 but the example notebooks use 7,500 to match production runs. Larger = better orbital library resolution.

6. Entry points and full signatures

6.1 `utils.load_data_bootstrap`

```
def load_data_bootstrap(path, filename, N_BOOTSTRAP=100, n_samples=5_000):
    # returns dict_data as in $5
```

6.2 `utils.logMenc_logc_to_logM_logRs, logM_logRs_to_logMenc_logc`

```
def logMenc_logc_to_logM_logRs(logM_enc, log_c,
                               r_enc=10.0, Delta=200., rho_crit=277.54):
    # (logM(<r_enc), log c) <-> (log M_NFW, log R_s)
def logM_logRs_to_logMenc_logc(logM_halo, logRs_halo,
                               r_enc=10.0, Delta=200., rho_crit=277.54):
    # inverse
```

Slot 0 of θ is `logM_10kpc`, slot 3 is `logc_halo` — the forward model wants (logM, logRs), so every wrapper starts with a call to `logMenc_logc_to_logM_logRs`.

6.3 `model.build_model` — forward model for χ^2 /likelihood

```
def build_model(density_func, potential_func,
                params_halo_pot, params_disk_rho, dict_data, num_Vbin,
                Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                Rzphi_lim_grid=jnp.array([[0,10],[-3,3],[-pi,pi]]),
                xy_lim_grid=jnp.array([[-10,10],[-3,3]]),
                xy_n_grid=jnp.array([60,40]),
                nnls_maxiter=200, regularization=1.0)
    -> dict[str, jnp.ndarray]
```

Returns a dict with these keys — all fields except `density_3d` and `weights` are **stacked over N_BOOTSTRAP bootstrap replicates** (row 0 = unperturbed):

key	shape	notes
<code>density_3d</code>	(Rzphi_n_tot,)	analytic 3-D density target (data)
<code>density_3d_model</code>	(N_boot, Rzphi_n_tot)	model 3-D density per bootstrap
<code>surface_density</code>	(N_boot, n_bins)	luminosity units
<code>V, sigma</code>	(N_boot, n_bins)	derived from h_1, h_2 via <code>h_to_V_sigma</code>
<code>h1..h4</code>	(N_boot, n_bins)	clipped to ± 10
<code>weights</code>	(N_boot, n_orb)	non-negative orbital weights

Static args (JIT recompiles when changed): `density_func, potential_func, num_Vbin, Rzphi_n_tot, nnls_maxiter.`

Time on A100: ~1 s per call after JIT warm-up. On CPU: ~60–100 s.

6.4 `model.get_model_with_orbit` — forward model + full orbit trajectories

```
def get_model_with_orbit(density_func, potential_func,
                        params_halo_pot, params_disk_rho, dict_data, num_Vbin,
                        Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                        Rzphi_lim_grid=jnp.array([[0,10],[-3,3],[-pi,pi]]),
                        xy_lim_grid=jnp.array([[-10,10],[-3,3]]),
                        xy_n_grid=jnp.array([60,40]),
                        nnls_maxiter=200, regularization=1.0)
    -> dict[str, jnp.ndarray]
```

Same forward model as `build_model`, but with `n_realizations=1` (one orbit per particle, no random jitter) and the integrator stores every accepted timestep's 6-D phase-space coordinates. Extra keys:

key	shape	notes
<code>weights</code>	<code>(n_particles,)</code>	unperturbed NNLS weights (row 0 of bootstrap)
<code>weights_all_boot</code>	<code>(N_boot, n_particles)</code>	per-bootstrap weights
<code>y_traj</code>	<code>(n_particles, 1, N_max, 6)</code>	<code>(x, y, z, vx, vy, vz)</code> in kpc, kpc/Gyr
<code>t_traj</code>	<code>(n_particles, 1, N_max)</code>	integration times in Gyr
<code>rotation_matrix</code>	<code>(3, 3)</code>	Z-X-Z rotation for viewing angles
<code>Omega_bar</code>	scalar	pattern speed in kpc/Gyr
<code>mass_per_orbit</code>	scalar	mean orbit mass (Msun)

Accepted timesteps are those with `dt = t[i+1]-t[i] > 0`. Padded positions (integrator finished before `N_max`) have `dt == 0` and should be masked out.

Same static-arg rules as `build_model`. Same runtime cost.

6.5 `model.model_deltaChi2_jackknife` — delete-d jackknife

```
def model_deltaChi2_jackknife(density_func, potential_func, chi2_func,
                             params_halo_pot, params_disk_rho, dict_data,
                             num_Vbin,
                             Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                             Rzphi_lim_grid=..., xy_lim_grid=..., xy_n_grid=...,
                             nnls_maxiter=200, regularization=1.0,
                             n_groups=100, batch_size=50)
    -> scalar (jnp.std of  $\chi^2$  across n_groups replicates)
```

`chi2_func` **must** have the signature:

```
def chi2_func(y_Rzphi, y_Rzphi_model, sigma_Rzphi,
             y_xy, y_xy_model, sigma_xy, # <-- ALL in luminosity units
             y_h1, y_h1_model, sigma_h1,
             y_h2, y_h2_model, sigma_h2,
             y_h3, y_h3_model, sigma_h3,
             y_h4, y_h4_model, sigma_h4)
    -> scalar
```

For each of `n_groups` jackknife replicates, the package raises the `sigma_*` entries of one disjoint bin-group to

1e20 (masking them from the NNLS), re-solves the NNLS for the replicate, computes `chi2_func`, and finally returns `std(chi2_replicate)`. Replicates are streamed through `jax.lax.map` in batches of `batch_size` so peak memory stays bounded regardless of `n_groups`.

Critical: see pitfall §8.4 for how `chi2_func` must handle `sigma_xy` (masking + your own error model).

6.6 `likelihood.model_likelihood` — canonical MCMC log-likelihood

```
def model_likelihood(params, dict_data, num_Vbin, Rzphi_n_tot,
                    sigma_amplify=1.0) -> scalar
```

Takes the 12-D θ , calls `build_model`, computes per-bootstrap χ^2 against Σ , h_1 – h_4 (and $Rzphi$ density), does log-mean-exp over bootstraps $\rightarrow$ `logL_marg`. Use this to drive an MCMC. `sigma_amplify` multiplies the χ^2 denominator (data errors) if you want an inflation factor.

6.7 `likelihood.calculate_delta_chi2` — jackknife wrapper

```
def calculate_delta_chi2(params, dict_data, num_Vbin, Rzphi_n_tot) -> scalar
```

Thin wrapper: unpacks $\theta \rightarrow$ dicts, calls `model_deltaChi2_jackknife` with a default 100-group jackknife. Read the source at `likelihood.py:249` to see the exact `chi2_func` it uses if you need to customise.

6.8 Rotation curve / `L_circ` helpers

```
def get_rotation_curve(R, potential_fn, potential_args=(),
                      z=0.0, dR=1e-3):
    # v_c(R) = sqrt(R * dPhi/dR) at phi = 0
    # returns kpc/Gyr — multiply by KPCGYR_TO_KMS for km/s
def build_Lcirc_of_E(potential_fn, potential_args=(),
                    R_min=1e-2, R_max=1e2, n_grid=2000, dR=1e-3):
    # returns (Lcirc_interp: callable, E_grid, L_grid)
```

`get_rotation_curve` is JIT'd and `jax-transformable`. `build_Lcirc_of_E` returns a `scipy interp1d` (host-side) — use it to compute circularity $\lambda_z = L_z / L_{\text{circ}}(E)$.

7. Common workflows

7.1 Convert θ to halo/baryon dicts (mandatory)

Every entry point except the likelihood wrappers expects two dicts, not the raw vector. Standard helper:

```
def unpack_params(theta):
    """12-D vector (angles in RAD) -> halo/baryon dicts (angles in DEG)."""
    (logM_10kpc, logM_disk, logM_bar, logc_halo,
     logRs_disk, logHs_disk, logL_bar,
     alpha_rad, beta_rad, gamma_rad,
     log_LM, log_Omega) = [float(x) for x in theta]
    logM_halo, logRs_halo = logMenc_logc_to_logM_logRs(
        logM_10kpc, logc_halo, r_enc=10.0, Delta=200., rho_crit=277.54)
    L_bar = 10.0 ** logL_bar
    Hs = 10.0 ** logHs_disk
    params_halo_pot = dict(
        logM=float(logM_halo), Rs=10.0**float(logRs_halo),
        a=1.0, b=1.0, c=1.0,
        x_origin=0.0, y_origin=0.0, z_origin=0.0,
```

```

    dirx=0.0, diry=0.0, dirz=1.0,
)
params_baryon_rho = dict(
    logM_disc=logM_disk, Rs_disc=10.0**logRs_disk, Hs_disc=Hs,
    logM_bar=logM_bar, L_bar=L_bar, a_bar=L_bar/5.0, b_bar=Hs,
    mass_to_light_ratio=10.0**log_LM,
    Omega_bar=10.0**log_Omega,
    x_origin=0.0, y_origin=0.0, z_origin=0.0,
    dirx=0.0, diry=0.0, dirz=1.0,
    alpha=alpha_rad * 180/np.pi, # DEG
    beta =beta_rad * 180/np.pi,
    gamma=gamma_rad * 180/np.pi,
)
return params_halo_pot, params_baryon_rho

```

Copy this verbatim into new code. It's canonical.

7.2 Define density and potential closures (mandatory)

```

from potentials import (NFW_potential,
                        MiyamotoNagai_potential, MiyamotoNagai_density)
from dehnen_bar import (T3_potential, T3_density,
                        V4_potential, V4_density)

V4_A, V4_B, V4_L, V4_GAMMA = 0.5, 0.5, 0.1, 0.0
GAMMA_BAR = 1.0

@jax.jit
def density_func(x, y, z, params):
    mn_params = {k: params[k] for k in
                  ('logM_disc', 'Rs_disc', 'Hs_disc',
                   'x_origin', 'y_origin', 'z_origin',
                   'dirx', 'diry', 'dirz')}
    rho_mn = MiyamotoNagai_density(x, y, z, mn_params)
    M_bar = 10.0 ** params['logM_bar']
    rho_t3 = T3_density(x, y, z, M_bar, params['a_bar'],
                       params['b_bar'], params['L_bar'], GAMMA_BAR)
    rho_v4 = V4_density(x, y, z, M_bar, V4_A, V4_B, V4_L, V4_GAMMA)
    return rho_mn + rho_t3 + rho_v4

@jax.jit
def potential_func(x, y, z, params_baryon, params_halo):
    phi_halo = NFW_potential(x, y, z, params_halo)
    mn_params = {k: params_baryon[k] for k in
                  ('logM_disc', 'Rs_disc', 'Hs_disc',
                   'x_origin', 'y_origin', 'z_origin',
                   'dirx', 'diry', 'dirz')}
    phi_mn = MiyamotoNagai_potential(x, y, z, mn_params)
    M_bar = 10.0 ** params_baryon['logM_bar']
    phi_t3 = T3_potential(x, y, z, M_bar, params_baryon['a_bar'],
                          params_baryon['b_bar'], params_baryon['L_bar'],
                          GAMMA_BAR)
    phi_v4 = V4_potential(x, y, z, M_bar, V4_A, V4_B, V4_L, V4_GAMMA)
    return phi_halo + phi_mn + phi_t3 + phi_v4

```

Again — copy verbatim. `V4_A`, `V4_B`, `V4_L`, `V4_GAMMA`, `GAMMA_BAR` are "soul-of-the-model" constants; do not change without deep understanding.

7.3 Forward model at one θ (for a plot)

```

p_halo, p_baryon = unpack_params(theta)
model_dict = build_model(
    density_func, potential_func,
    p_halo, p_baryon, dict_data,
    num_Vbin=dict_data['total_bins'],
    Rzphi_n_tot=dict_data['Rzphi_n_tot'],
    Rzphi_n_grid=dict_data['Rzphi_n_grid'],
    Rzphi_lim_grid=jnp.array([dict_data['R_minmax'],
                              dict_data['z_minmax'],
                              dict_data['phi_minmax']]),
    xy_lim_grid=jnp.array([dict_data['X_minmax'],
                           dict_data['Y_minmax']]),
    xy_n_grid=jnp.array(dict_data['nX_nY']),
)
# Take row 0 (unperturbed) for plotting:
density_model = np.asarray(model_dict['surface_density'][0])
V_model       = np.asarray(model_dict['V'][0])
h1_model      = np.asarray(model_dict['h1'][0])
# ... etc

```

Model surface density is in **the same luminosity units** as `dict_data['XY_density_data']`. Compare directly.

7.4 Orbital library with trajectories

```

p_halo, p_baryon = unpack_params(theta_bestfit)
orb = get_model_with_orbit(density_func, potential_func,
                           p_halo, p_baryon, dict_data,
                           num_Vbin=dict_data['total_bins'], ...)

# Every accepted timestep = one weighted particle sample.
weights = np.asarray(orb['weights'])           # (n_particles,)
y_traj  = np.asarray(orb['y_traj'])           # (n_particles, 1, N_max, 6)
t_traj  = np.asarray(orb['t_traj'])
dt      = np.diff(t_traj[:, 0], axis=1, prepend=t_traj[:, 0, :1]) # (n_particles, N_max)
accepted = dt > 0
y_flat  = y_traj[:, 0][accepted]               # (N_accepted, 6)
w_flat  = (dt * weights[:, None])[accepted]    # (N_accepted,) = dt * orbit weight

# Apply 4-fold bar symmetry:
SIGN_SYM = np.array([
    [ 1, 1, 1, 1, 1, 1], [ 1, 1,-1, 1, 1,-1],
    [-1,-1, 1,-1,-1, 1], [-1,-1,-1,-1,-1,-1],
], dtype=np.float32)
y_sym = (y_flat[None] * SIGN_SYM[:, None, :]).reshape(-1, 6)
w_sym = np.tile(w_flat, 4)

```

`example_analysis.ipynb` Section 4 (in the example folder) shows the full pipeline down to a circularity histogram.

7.5 MCMC driver skeleton

Not fully unrolled here — see `example/example_schwarmax.ipynb` for the canonical BlackJAX adaptive-RMH driver. The critical thing is that the MCMC calls `model_likelihood` (or a very close variant of it) and does **not** call `build_model` directly. The likelihood wrapper does the θ -unpacking; the MCMC just sees a scalar function of θ .

7.6 Jackknife error bar

```

def chi2_func(y_Rzphi, y_Rzphi_model, sigma_Rzphi,
              y_xy,    y_xy_model,    sigma_xy,    # in luminosity units

```



```

        y_h1,    y_h1_model,    sigma_h1,
        y_h2,    y_h2_model,    sigma_h2,
        y_h3,    y_h3_model,    sigma_h3,
        y_h4,    y_h4_model,    sigma_h4):
# Respect the mask: replicates raise sigma_xy to 1e20 on dropped bins.
# Override to 1% relative error ONLY on kept bins:
sig_xy = jnp.where(sigma_xy < 1e10, 0.01 * y_xy, sigma_xy)
res_xy = jnp.nansum(((y_xy - y_xy_model) / sig_xy) ** 2)
res_h1 = jnp.nansum(((y_h1 - y_h1_model) / sigma_h1) ** 2)
# ... etc for h2, h3, h4
return -0.5 * (res_xy + res_h1 + res_h2 + res_h3 + res_h4)

delta_chi2 = model_deltaChi2_jackknife(
    density_func, potential_func, chi2_func,
    p_halo, p_baryon, dict_data,
    num_Vbin=dict_data['total_bins'], ...,
    n_groups=100, batch_size=25)

```

8. Pitfalls — READ THIS

8.1 The `log_mass_to_light` sign convention

This is the single biggest footgun.

The **older** `SchwarMAX/` codebase defines slot 10 of θ as $\log(L/M)$ and pipes `light_to_mass_ratio = 10**theta[10]` into the forward model.

The **newer** `schwarmax_pkg/` (this package) defines slot 10 of θ as $\log(M/L)$ and pipes `mass_to_light_ratio = 10**theta[10]`. Inside `build_model` it inverts internally.

Same numerical value in slot 10 → **opposite physical meaning** in the two pipelines.

Consequence for old checkpoints: 13-D chains produced by the old `likelihoods_bar.py` need `theta[10] → -theta[10]` before feeding them to `build_model`. 12-D chains produced by `likelihood.model_likelihood` are already in the correct convention and do **not** need flipping.

`example_analysis.ipynb` auto-detects by `ndim` (13 → flip, 12 → don't). If you are writing new code that consumes an existing checkpoint: **check the shape** and flip accordingly.

8.2 Angles are radians in θ , degrees in the baryon dict

`build_model` reads `params_baryon['alpha']` in degrees and calls `makeRotationMatrix(alpha, beta, gamma)` which expects degrees. But the MCMC chain stores angles in **radians**. `unpack_params` (§7.1) converts. If you build the baryon dict by hand, remember to multiply by `180/np.pi`.

8.3 `build_model` returns bootstrap-stacked arrays

Every kinematic/density field is `(N_BOOTSTRAP, n_bins)`. Row 0 is the unperturbed data (`XY_standard_normal[0]` is zeroed in `load_data_bootstrap`), so `model_dict['surface_density'][0]` is your best-fit model for plotting.

8.4 Jackknife `chi2_func` — units and masking

Units: `model_deltaChi2_jackknife` passes `y_xy` and `sigma_xy` to your `chi2_func` in **luminosity units** (fixed in commit 5f59b9f onwards). Before that fix, `y_xy` was in internal NNLS units while `y_xy_model` was in luminosity — the resulting χ^2 was garbage.

Masking: the package's per-replicate masking works by raising `sigma_xy` to $1e20$ on dropped bins. If your `chi2_func` **overrides** `sigma` unconditionally (e.g. `sig_xy = 0.01 * y_xy`), the mask is lost and dropped bins are counted with a tiny error, inflating the jackknife std. The correct pattern:

```
sig_xy = jnp.where(sigma_xy < 1e10, 0.01 * y_xy, sigma_xy)
```

8.5 Kinematic bins are already un-normalised via `gamma_kin`

Model h-moments are computed as $(A_{hk} @ w) / \gamma_{kin}$ where $\gamma_{kin} = M_{per_bin} / (norm \cdot 2\sqrt{\pi} \cdot s)$ (Agama's option-b convention). The data h-moments are `dict_data['h1_data']` etc., not multiplied by anything. **Do not** divide the model h-moments by the surface density; that would double-normalise.

8.6 JIT static args and recompilation

The following args of `build_model` are static, meaning JAX recompiles when they change:

- `density_func`, `potential_func` (structural — normally frozen at module load)
- `num_Vbin`, `Rzphi_n_tot`, `nnls_maxiter`

Changing `n_samples` in `load_data_bootstrap` changes the number of particles → different `w0.shape[0]` → triggers a recompile. If you sweep many parameter vectors, keep `dict_data` fixed.

Also: don't pass Python floats where jax expects arrays (or vice versa). `params_halo_pot['logM']` must be a **Python float**, not a `jnp.float32`, to avoid re-tracing per posterior sample.

8.7 The N-body snapshot is 126 MB and Git-LFS-tracked

`example/Bar_model_TG21/model/t_t0_7` is stored in Git LFS. After cloning:

```
git lfs install      # one-time setup
git lfs pull         # actually download the snapshot
```

If you skip these, the file will be a 134-byte pointer and any snapshot-using cell will fail with an obscure error.

8.8 The `example_schwarmax.ipynb` cell 1 assumes Google Colab

The first cell does `from google.colab import drive` and pip-installs `jaxopt / corner / emcee / blackjax`. If you're running the notebook locally, replace this cell with a `sys.path.insert(0, "..")` and skip the pip installs (or run them via `pip install` manually).

8.9 GPU is highly recommended for MCMC

`build_model` on CPU takes ~60–100 s per call. For 16 chains × 3000 steps that's ~20 days of CPU time. On an A100 the same run is ~4 h. For **analysis** (single- θ or small posterior sweeps) CPU is fine.

9. The example folder

The example folder is the ground truth for how to use the package.

- `example_schwarmax.ipynb` — Colab-friendly BlackJAX adaptive RMH driver. Loads `mock_Nbody_galaxy_beta65_gamma45_D50.pkl` via `load_data_bootstrap`, defines `density_func` + `potential_func` + `model_likelihood`, runs the MCMC, checkpoints every 5 steps.

- `example_analysis.ipynb` — 4-section post-MCMC tutorial:

1. Corner plot (posterior visualisation).
2. Data vs model in the disc-aligned frame (uses `build_model`).
3. Rotation curve + DM enclosed mass recovery (uses `get_rotation_curve` and analytic NFW; compares to N-body ground truth via AGAMA).
4. Orbital structure — circularity histogram from `get_model_with_orbit` vs N-body.

- `mcmc_checkpoint.pk1` — 12-D chain, 1000 steps × 32 chains, produced by `example_schwarmax.ipynb`.
- `mock_Nbody_galaxy_beta65_gamma45_D50.pk1` — Voronoi-binned mock IFU data (Σ , V , σ , h_1 – h_4 + errors + orientation) at $\alpha=25^\circ$, $\beta=65^\circ$, $\gamma=45^\circ$, distance 50 Mpc.
- `Bar_model_TG21/model/t_t0_7` — raw N-body snapshot (126 MB, Git LFS). Used for ground-truth density contours, halo $M(<r)$, and circularity.
- `Bar_model_TG21/model/t_t0_7.ini` — pre-computed AGAMA CylSpline potential from the snapshot. Loading is fast; use this for $v_c(R)$ and $L_{\text{circ}}(E)$ on the N-body side.

10. Where to look for canonical usage of each function

Function	Canonical use
<code>load_data_bootstrap</code>	<code>example_analysis.ipynb</code> Section 0, cell 5
<code>build_model</code>	<code>example_analysis.ipynb</code> Section 2, cell 11
<code>get_model_with_orbit</code>	<code>example_analysis.ipynb</code> Section 4, cell 22
<code>model_deltaChi2_jackknife</code>	there is no notebook demo yet — see <code>temp/jackknife_problem.py</code> at the repo root for a minimal driver
<code>model_likelihood</code>	<code>example_schwarmax.ipynb</code> cell 2 (defines wrapper), cell 5 (calls it to probe likelihood slices)
<code>get_rotation_curve</code>	<code>example_analysis.ipynb</code> Section 3, cell 15
<code>build_Lcirc_of_E</code>	<code>example_analysis.ipynb</code> Section 4, cell 23
<code>logMenc_logc_to_logM_logRs</code>	inside <code>unpack_params</code> — <code>example_analysis.ipynb</code> Section 0, cell 4

11. What to do when the user reports a bug

Before "fixing" anything:

1. Ask which pipeline generated the checkpoint (12-D vs 13-D — §8.1).
2. Confirm the parameter vector matches §4 slot-by-slot.
3. Check whether the user hand-crafted `dict_data` (they shouldn't; §5).
4. Check units of any custom `chi2_func` (§8.4).
5. Ask whether `t_t0_7` was pulled from LFS (§8.7).

Most reported "bugs" are unit or convention issues in the caller. Actual bugs in the package are rare — the code is short and heavily peer-reviewed via the example notebook.

12. Contact

If you get truly stuck, the source is small (~2,500 LOC of jax across the 12 package files) and every function has a short docstring or inline comment. `git log --oneline` shows a clean history; `git blame` each critical line to

see who added it and why. Bar-branch commits from mid-2026 (this repo) are the most recent — earlier commits in the sibling `SchwarMAX/` codebase are archaeology and should be treated as such.

End of guide.